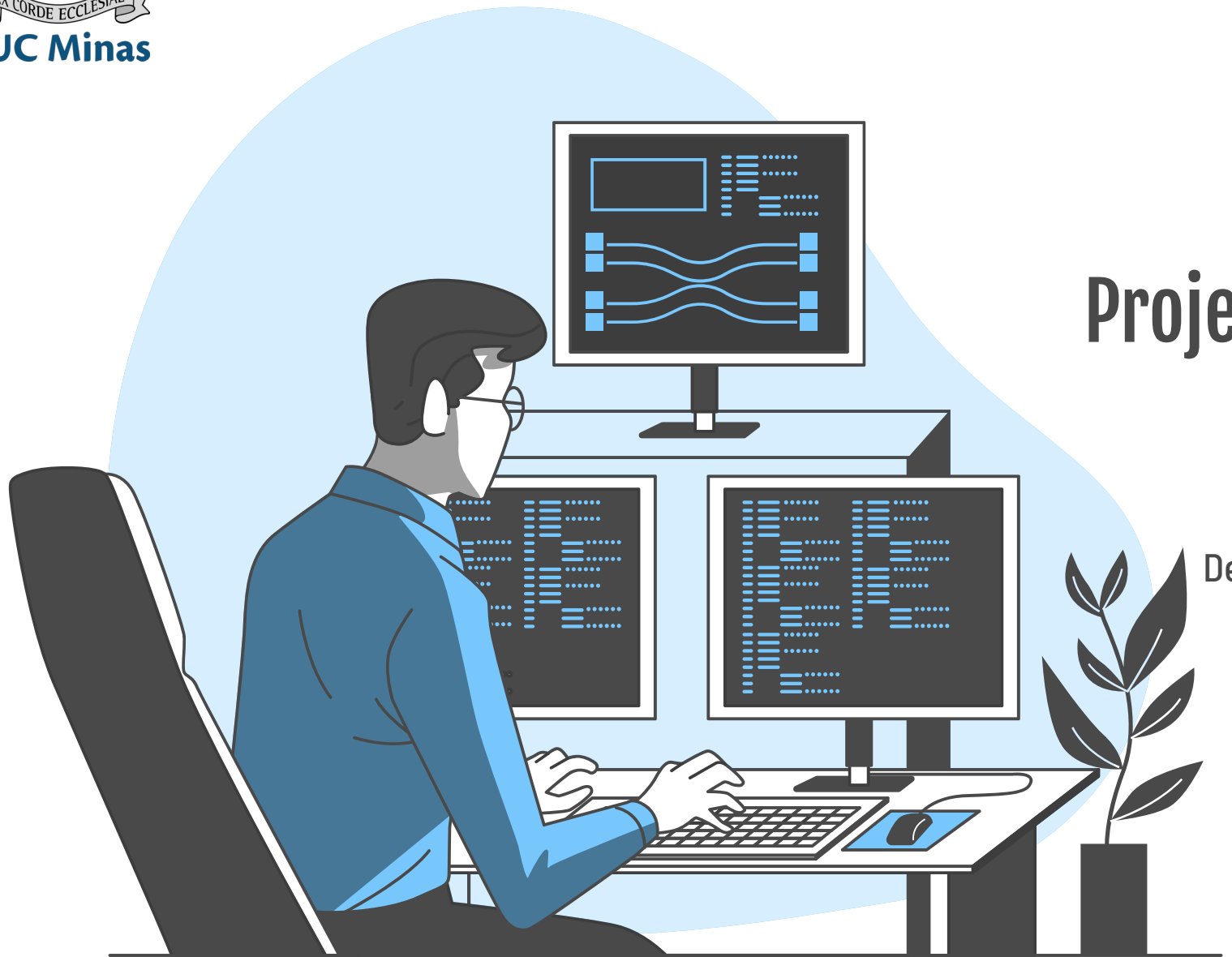
PUC Minas



Aula 09 · Front-end

# Projeto prático: React + Vite + Supabase

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

# Nesta aula você vai

1. Criar um projeto gratuito no **Supabase**
2. Criar uma tabela no Supabase pelo **SQL Editor**
3. Entender a **Data API** e a **publishable key**
4. Proteger a tabela com **RLS** (*Row Level Security*) e *policies*
5. Usar o `@supabase/supabase-js` para listar, inserir, atualizar e excluir
6. Montar o projeto com `config/`, `repository/` e componentes

# Antes de começar

- Aulas 05 a 07
- Conta no Supabase (supabase.com, login com GitHub)

Duração sugerida

**100 min**

**Código da disciplina**

Projetos de referência na pasta PROJETOS do GitHub

# Roteiro


01 O que é o Supabase

02 Duas formas de usar o Supabase

03 Passo 0: criar o projeto no Supabase

04 Passo 1: criar o projeto React

05 Passo 2: criar a tabela e as regras de acesso

06 Passo 3: copiar URL e chave publicável

07 Passo 4: o código

08 Passo 5: testar e conferir no painel

09 Aprofundando: login com Supabase Auth

01

# O que é o Supabase

# O que é o Supabase

O **Supabase** é uma plataforma que entrega um **banco PostgreSQL completo na nuvem**, com painel web, editor de tabelas, editor SQL, autenticação, armazenamento de arquivos e uma **API gerada automaticamente** para cada tabela. O plano gratuito permite 2 projetos ativos, suficientes para todos os trabalhos da disciplina.

01

**O que é o Supabase**

Aula 09 · Front-end

02

# Duas formas de usar o Supabase

# Duas formas de usar o Supabase

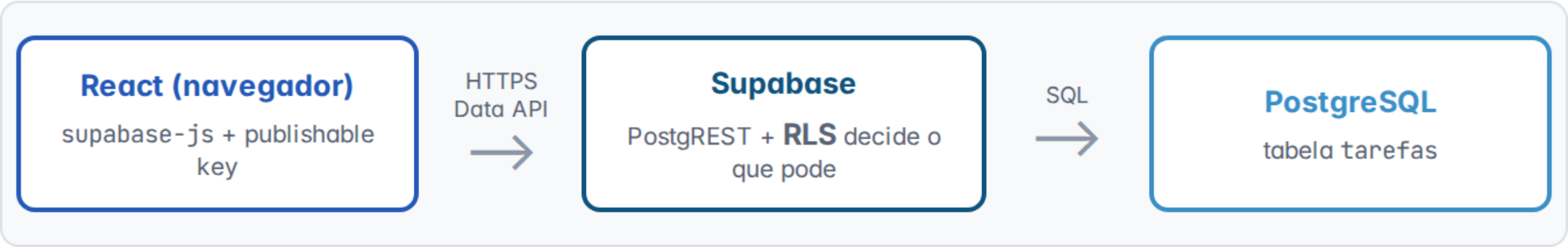
Mais adiante, na pasta Back-end, um **Spring Boot** vai se conectar a esse mesmo PostgreSQL via JDBC. Nesta aula o **navegador** fala com o Supabase diretamente, sem back-end próprio. Como o navegador não consegue abrir conexão TCP com o Postgres, o `supabase-js` usa a **Data API** (HTTPS) que o Supabase gera automaticamente para cada tabela.

02

**Duas formas de usar o Supabase**

Aula 09 · Front-end

# Duas formas de usar o Supabase




Sem back-end próprio: quem protege os dados é o **RLS** no banco, não a chave.

# Duas formas de usar o Supabase

É assim que funciona o projeto `React/Pokedex` (favoritos salvos no Supabase). Para dados sensíveis, regras complexas ou chaves secretas, use um back-end Spring Boot (pastas Back-end e FullStack).

02

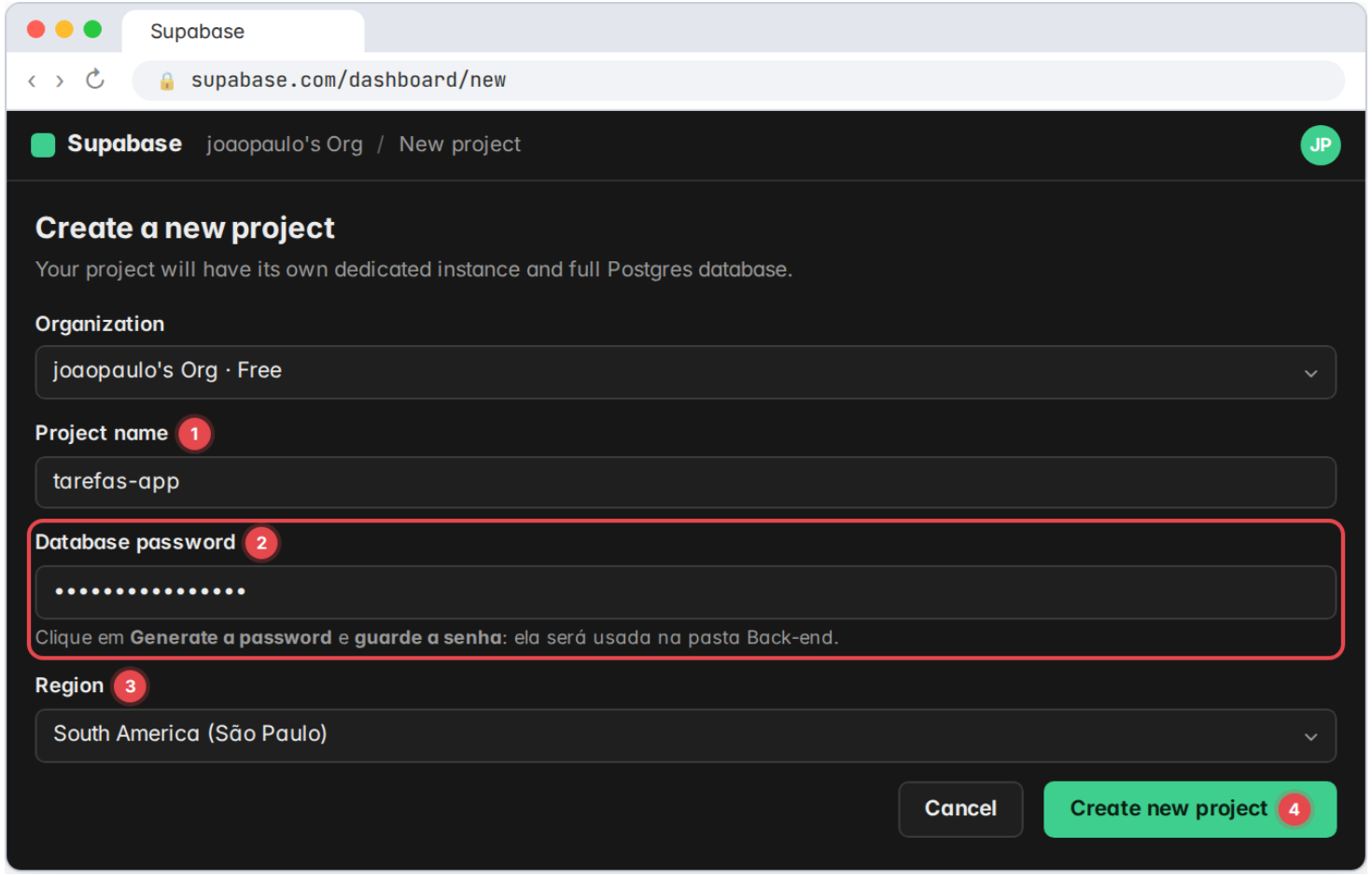
**Duas formas de usar o Supabase**

Aula 09 · Front-end

# 03

# Passo 0: criar o projeto no Supabase

# Passo 0: criar o projeto no Supabase




Criação de um projeto no Supabase.

# Passo 0: criar o projeto no Supabase

1. Acesse **supabase.com**, entre com o GitHub e clique em **New project**. Dê um nome ao projeto (`tarefas-app`).
2. Gere a **senha do banco** e guarde em um lugar seguro. Nesta aula ela não é usada (o navegador usa a chave publicável), mas o Spring Boot vai precisar dela na pasta Back-end.
3. Escolha a região **São Paulo** (menor latência para o Brasil).
4. Clique em **Create new project** e aguarde 1 a 2 minutos enquanto o banco é provisionado.

# Passo 0: criar o projeto no Supabase

**Atenção**

No plano gratuito, projetos sem uso por **7 dias** são pausados. Basta entrar no painel e clicar em **Restore project**. Lembre disso antes de apresentar um trabalho!

# 04

# Passo 1: criar o projeto React

# Passo 1: criar o projeto React

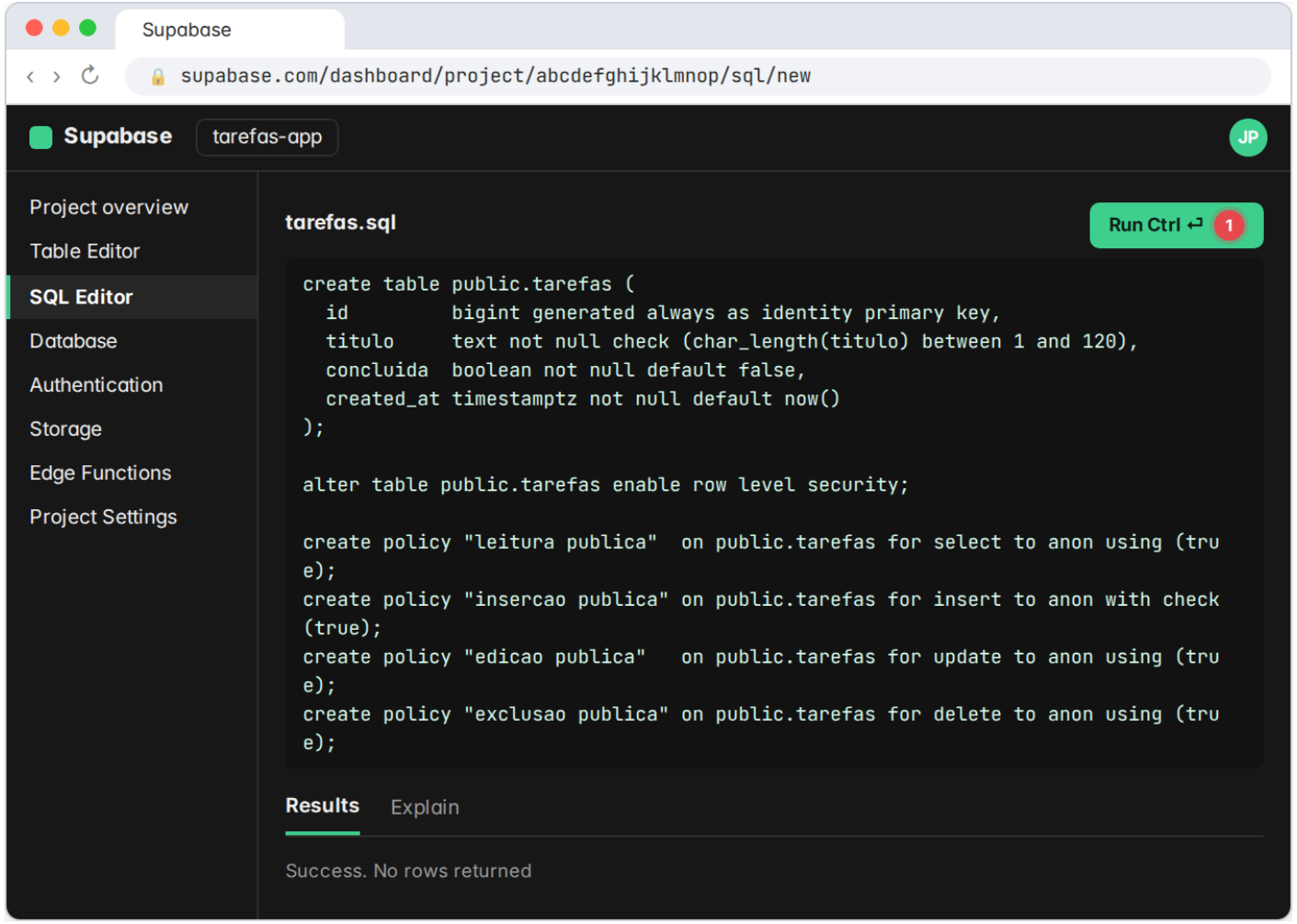
Terminal

```
npm create vite@latest tarefas -- --template react
cd tarefas
npm install
npm install @supabase/supabase-js
npm run dev
```

05

# Passo 2: criar a tabela e as regras de acesso

# Passo 2: criar a tabela e as regras de acesso




Criando a tabela, ligando o RLS e liberando as operações para o papel `anon` (visitante).

# Passo 2: criar a tabela e as regras de acesso

- **RLS ligado e sem policies = ninguém acessa** pela Data API. Cada `create policy` libera uma operação.
- `to anon`: vale para quem usa a chave publicável sem login.
- `using (true)`: todas as linhas. Em um app com login, seria `using (auth.uid() = user_id)`, e cada usuário veria só as próprias tarefas.

05

**Passo 2: criar a tabela e as regras de acesso**

Aula 09 · Front-end

# Passo 2: criar a tabela e as regras de acesso

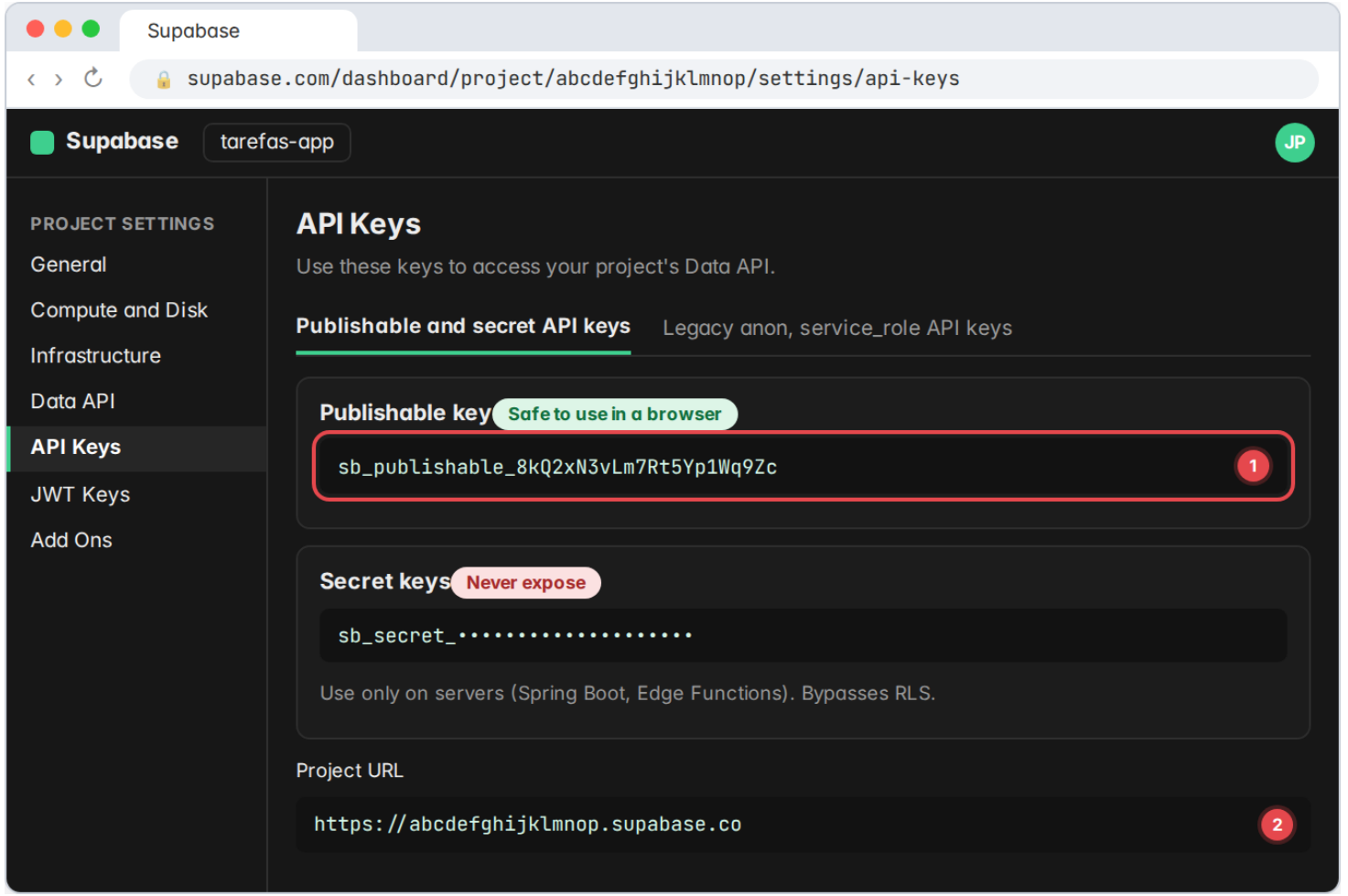
### Atenção

As policies acima deixam **qualquer visitante** ler, criar, alterar e apagar tarefas: ótimo para aprender, inadequado para produção. O projeto `Pokedex` mostra uma alternativa mais segura: RLS ligado, **nenhuma** permissão direta na tabela, e o navegador só chama **funções SQL** (`list_my_favorites`, etc.) que validam os dados.

06

# Passo 3: copiar URL e chave publicável

# Passo 3: copiar URL e chave publicável




Em **Project Settings → API Keys**, copie a **publishable key**; a URL fica em **Data API** (ou no botão **Connect → App Frameworks**).

# Passo 3: copiar URL e chave publicável

.env.local (na raiz do projeto)

```
VITE_SUPABASE_URL=https://abcdefghijklmnop.supabase.co
VITE_SUPABASE_PUBLISHABLE_KEY=sb_publishable_8kQ2xN3vLm7Rt5Yp1Wq9Zc
```

# Passo 3: copiar URL e chave publicável

## Erro comum

A **secret key** (ou a antiga `service_role`) **ignora o RLS**: quem a tiver pode apagar o banco inteiro. Ela **nunca** vai em variável `VITE_`. Se vazou, gere outra imediatamente em **API Keys**.

07

# Passo 4: o código

# Passo 4: o código

src/config/supabaseClient.js

```
import { createClient } from "@supabase/supabase-js";

const url = import.meta.env.VITE_SUPABASE_URL;
const key = import.meta.env
  .VITE_SUPABASE_PUBLISHABLE_KEY;

export const supabase = createClient(url, key, {
  auth: { persistSession: false },
});
```

src/repository/tarefaRepository.js

```
import { supabase } from "../config/supabaseClient";

const T = "tarefas";

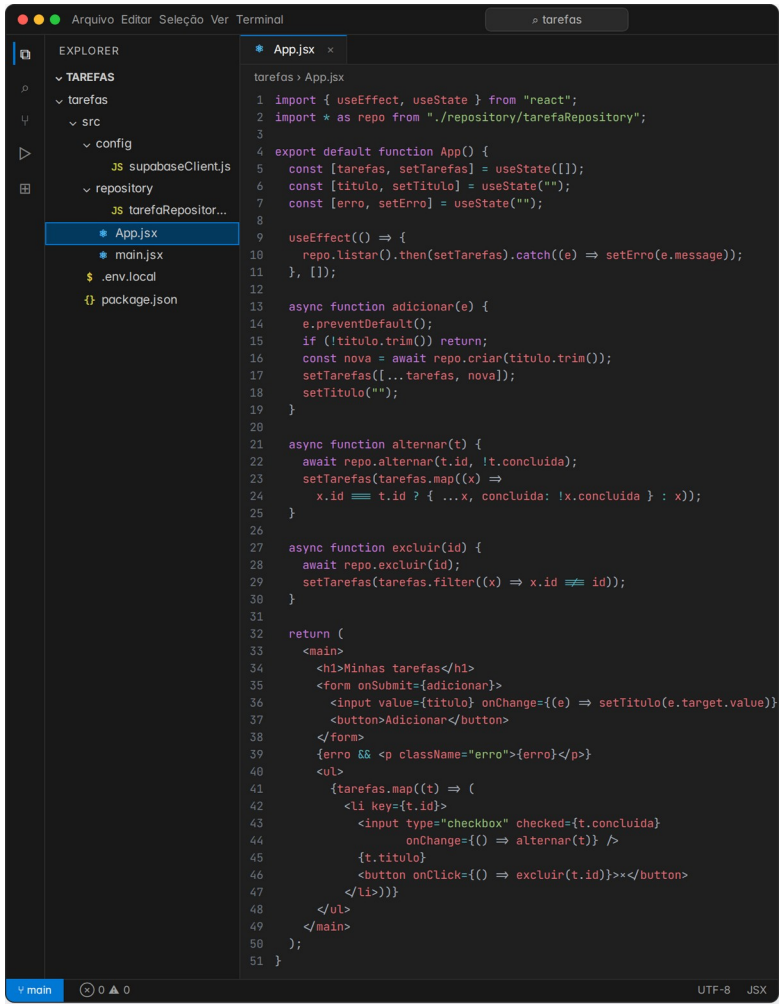
export async function listar() {
  const { data, error } = await supabase
    .from(T).select("*")
    .order("created_at", { ascending: true });
  if (error) throw error;
  return data;
}

export async function criar(titulo) {
  const { data, error } = await supabase
    .from(T).insert({ titulo }).select().single();
  if (error) throw error;
  return data;
}

export async function alternar(id, concluida) {
  const { error } = await supabase
    .from(T).update({ concluida }).eq("id", id);
  if (error) throw error;
}

export async function excluir(id) {
  const { error } = await supabase
    .from(T).delete().eq("id", id);
  if (error) throw error;
}
```

# Passo 4: o código

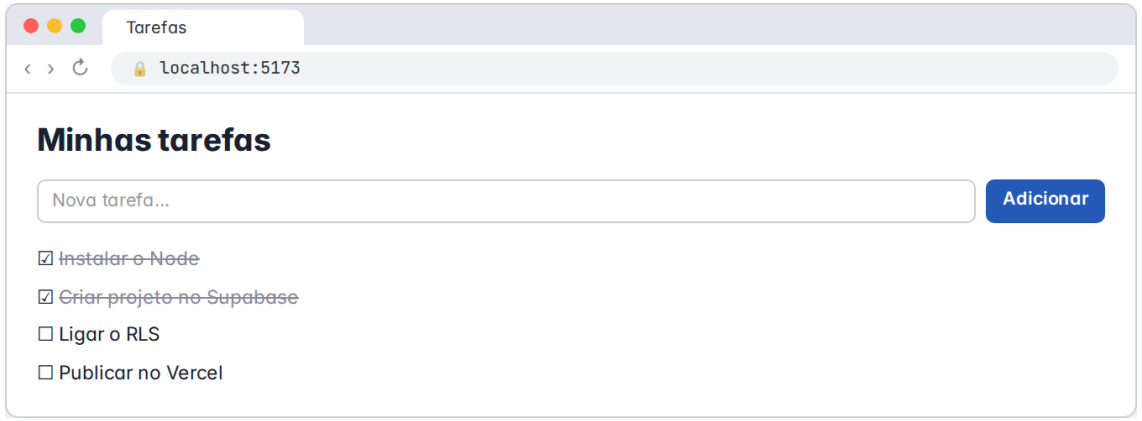



`App.jsx` completo do app de tarefas.

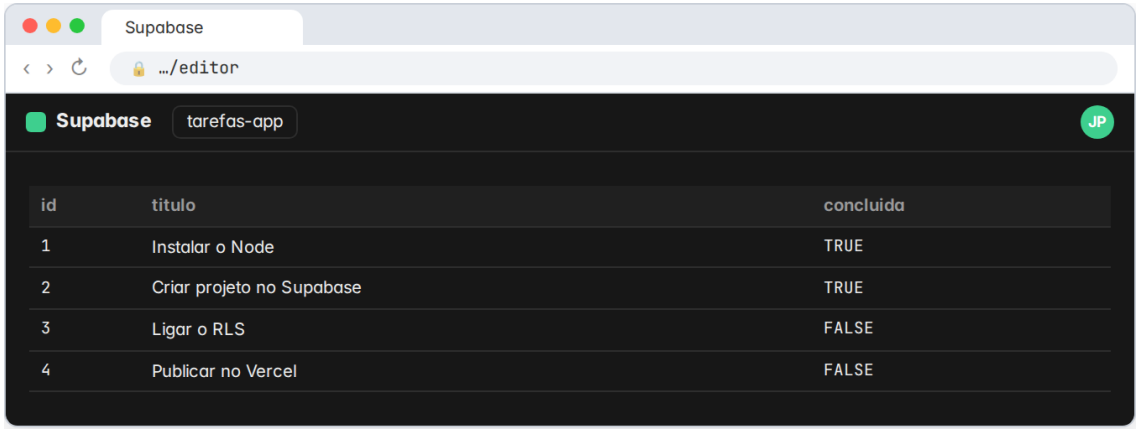
08

# Passo 5: testar e conferir no painel

# Passo 5: testar e conferir no painel




O app rodando.




Os mesmos dados no Table Editor.

# Passo 5: testar e conferir no painel

## Erro comum

`new row violates row-level security policy`: o RLS está ligado mas falta a policy da operação (ex.: `insert`). **Lista vem vazia sem erro**: falta a policy de `select`. `Invalid API key`: a chave no `.env.local` está errada ou o servidor não foi reiniciado.

# Passo 5: testar e conferir no painel

## Dica

Pronto para publicar? O deploy no Vercel é o mesmo de qualquer projeto Vite: cadastre `VITE_SUPABASE_URL` e `VITE_SUPABASE_PUBLISHABLE_KEY` nas variáveis de ambiente do Vercel (Deploy, aula 05).

09

# Aprofundando: login com Supabase Auth

# Aprofundando: login com Supabase Auth

Com o **Supabase Auth**, o próprio Supabase cadastra usuários e emite um JWT. As policies do RLS passam a usar `auth.uid()`, e cada usuário enxerga só as próprias linhas, ainda sem back-end próprio.

Policies por usuário

```
alter table public.tarefas add column user_id uuid references auth.users default auth.uid();

drop policy if exists "leitura publica" on public.tarefas;
create policy "dono le"     on public.tarefas for select to authenticated using (auth.uid() = user_id);
create policy "dono cria"   on public.tarefas for insert to authenticated with check (auth.uid() = user_id);
create policy "dono altera" on public.tarefas for update to authenticated using (auth.uid() = user_id);
create policy "dono apaga"  on public.tarefas for delete to authenticated using (auth.uid() = user_id);
```

# Aprofundando: login com Supabase Auth

supabase-js: autenticação

```
// cadastro e login por e-mail e senha
await supabase.auth.signUp({ email, password });
const { error } = await supabase.auth.signInWithPassword({ email, password });

// usuário atual e mudanças de sessão
const { data: { user } } = await supabase.auth.getUser();
supabase.auth.onAuthStateChange((_evento, sessao) => setSessao(sessao));

await supabase.auth.signOut();
```

# Aprofundando: login com Supabase Auth

### Observação

Para usar o Auth, remova `persistSession: false` do `createClient`: é a sessão salva no navegador que mantém o usuário logado. Ative o provedor *Email* em **Authentication → Sign In / Providers** no painel.

# Realtime: a lista atualiza sozinha

Outra aba criou uma tarefa? Esta aba recarrega

```
useEffect(() => {
  const canal = supabase
    .channel("tarefas-mudancas")
    .on("postgres_changes", { event: "*", schema: "public", table: "tarefas" },
        () => repo.listar().then(setTarefas))
    .subscribe();
  return () => supabase.removeChannel(canal);
}, []);
```

# Storage: upload de imagens

Bucket "imagens" criado em Storage

```
const caminho = `fotos/${crypto.randomUUID()}-${arquivo.name}`;
const { error } = await supabase.storage.from("imagens").upload(caminho, arquivo);
const { data } = supabase.storage.from("imagens").getPublicUrl(caminho);
// data.publicUrl → use no <img src>
```

# Supabase direto do front x Spring Boot no meio

## Direto (supabase-js)

- Menos código: sem back-end
- Segurança depende do RLS bem escrito
- Ótimo para protótipos e apps simples

vs

## Spring Boot + Supabase (JDBC)

- Regras em Java, testáveis
- Segredos ficam no servidor
- Padrão dos projetos FullStack

# Aprofundando: login com Supabase Auth

| Erro do supabase-js | Causa |
|---|---|
| `new row violates row-level security policy` | Falta a policy de insert/update |
| Lista vazia sem erro | Falta a policy de select (ou o usuário não é o dono) |
| `Invalid API key` | Chave errada no `.env.local` ou servidor não reiniciado |
| `relation "public.tarefas" does not exist` | Tabela não criada ou nome diferente |
| `JWT expired` | Sessão expirada: chame `getUser()`/faça login de novo |

10

# Quiz de revisão

# Pergunta

## Como o navegador acessa o Supabase sem back-end próprio?

A Conexão TCP direta ao Postgres

B Pela Data API (HTTPS) com supabase-js

C Por FTP

D Pelo Render

# Resposta

## Como o navegador acessa o Supabase sem back-end próprio?

A Conexão TCP direta ao Postgres

✓ Pela Data API (HTTPS) com supabase-js

C Por FTP

D Pelo Render

**Por quê:** O navegador não abre conexões TCP com o banco.

# Pergunta

## Com RLS ligado e nenhuma policy, o que acontece?

A Tudo liberado

B Nada passa pela Data API

C Só leitura

D Só inserção

# Resposta

## Com RLS ligado e nenhuma policy, o que acontece?

A Tudo liberado

✓ Nada passa pela Data API

C Só leitura

D Só inserção

**Por quê:** Cada policy libera uma operação.

Quiz · questão 3 de 5

# Pergunta

## Qual chave pode ir no `.env.local` do front?

A Secret key

B service_role

C Publishable key

D Senha do banco

# Resposta

## Qual chave pode ir no `.env.local` do front?

A. Secret key

B. service_role

✓ Publishable key

D. Senha do banco

**Por quê:** A secret ignora o RLS.

# Pergunta

## Qual erro aparece quando falta a policy de insert?

A Invalid API key

B new row violates row-level security policy

C CORS

D 404

# Resposta

## Qual erro aparece quando falta a policy de insert?

A Invalid API key

✓ new row violates row-level security policy

C CORS

D 404

**Por quê:** Crie a policy de insert para o papel adequado.

# Pergunta

## Em `supabase.from('tarefas').delete().eq('id', 5)`, o `.eq` faz:

A Ordena

B Filtra a linha com id = 5

C Conta

D Cria a tabela

# Resposta

## Em `supabase.from('tarefas').delete().eq('id', 5)`, o `.eq` faz:

A Ordena

✓ Filtra a linha com id = 5

C Conta

D Cria a tabela

**Por quê:** Sem filtro o delete seria recusado.

Fechamento

# Resumo da aula

- ✓ O navegador acessa o Supabase pela **Data API** usando `supabase-js` e a **publishable key**.
- ✓ Quem protege os dados é o **RLS** + *policies*; sem policy, nada passa.
- ✓ A **secret key** ignora o RLS e nunca vai para o front.
- ✓ Organização: `config/supabaseClient.js` → `repository/` → componentes.
- ✓ `supabase.from(tabela).select/insert/update/delete` + `.eq(...)`, sempre checando `error`.

# Exercícios

1. Construa o app de tarefas e publique no Vercel.

2. Adicione um filtro “todas / pendentes / concluídas” usando `.eq("concluida", ...)`.

3. Remova a policy de `delete` e observe o erro no console. Depois recoloque.

4. Estude o `supabase_favorites.sql` do projeto `Pokedex` e explique por que ele é mais seguro que as policies desta aula.

# Para ir além

- supabase.com/docs/reference/javascript
- supabase.com/docs/guides/database/postgres/row-level-security
- Projeto `PROJETOS/React/Pokedex`

### Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

*Links e versões citados podem ter mudado. Confira sempre a documentação oficial.*




# Obrigado!

Dúvidas sobre a aula 09?

**Prof. Dr. João Paulo Aramuni**
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

**GitHub da disciplina**
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web